integration_sec04

4. Phase 3: Scoring Engine and Score Adapter

The scoring engine constitutes the analytical core of the LLMsVerifier integration, transforming raw benchmark telemetry into actionable, ranked intelligence that HelixTranslate can consume for provider selection. This chapter documents the complete implementation of the `ScoringEngine` type, its five-component weighted scoring model, the composite score aggregation pipeline, the `ScoreAdapter` service that bridges LLMsVerifier's internal score representation to HelixTranslate's provider metadata model, and the persistent history subsystem that enables longitudinal trend analysis. Phase 3 is the longest-running stage of the integration and must be designed for both throughput — processing hundreds of score recalculations per evaluation cycle — and operational longevity, maintaining multi-year score archives without degradation to the translation system's cold-path latency.

4.1 Scoring Engine Initialization

The `ScoringEngine` is instantiated once at application startup and bound to the shared SQLite database handle that Phase 1 established. Its lifecycle is managed by the dependency-injection container defined in `cmd/helix-translate/main.go`, where the engine is registered as a singleton service under the interface `verifier.ScoringProvider`. The constructor receives four dependencies: the database connection (`*sql.DB`), an HTTP client for outbound benchmark requests, a `ScoreWeights` value object, and a structured logger. Each dependency is validated before assignment, and the constructor returns a descriptive error if any invariant is violated.

4.1.1 Configuration Options

The `ScoreWeights` struct and its surrounding configuration are populated from the `[verifier.scoring]` TOML stanza in `config.toml`. The scoring subsystem supports two operational profiles: `"standard"` and `"enhanced"`. In `"standard"` mode, scores are calculated directly from the latest benchmark run with no historical blending. In `"enhanced"` mode, the engine applies a seven-day exponential moving average (EMA) to each component before computing the composite, smoothing transient spikes caused by provider-side rate-limiting or temporary capacity

reductions. HelixTranslate deployments default to `"enhanced"` mode in production because translation workloads are cost-sensitive and require stable rankings over multi-day horizons.

The full configuration surface is itemised in **Table 1**.

Table 1 — Scoring Engine Configuration Options

Option	Type	Default	Description
Mode	string	`"standard"`	`"standard"` or `"enhanced"`; enhanced enables 7-day EMA smoothing
WeightSpeed	float64	`0.20`	Response speed weight in composite calculation
WeightCost	float64	`0.30`	Cost-effectiveness weight; highest in HelixTranslate profile
WeightEfficiency	float64	`0.25`	Token-efficiency weight; output-per-input ratio
WeightCapability	float64	`0.20`	Capability weight; challenge pass-rate normalised
WeightRecency	float64	`0.05`	Model recency weight; exponential age decay
CacheTTL	duration	`1h`	In-memory positive-score cache TTL
DBPath	string	`./data/verifier.db`	Path to the score-history SQLite database

The weight values in **Table 1** reflect the HelixTranslate-optimised profile, not the generic LLMsVerifier defaults. The rationale for this redistribution — reducing the emphasis on raw speed and recency while elevating cost and efficiency — is explored in detail in Section 4.2. The `CacheTTL` controls how long a fully resolved `CompositeScore` remains valid in the process-local LRU cache. A one-hour TTL provides a practical balance: short enough to react to daily provider price changes or model deprecations, long enough to avoid redundant database queries during high-volume translation batches that may invoke `Adapt` dozens of times per document.

4.1.2 Engine Constructor

The constructor in **Code Block 1** performs three critical initialisation steps. First, it invokes `validateWeights`, which enforces the mathematical invariant that the five component weights must sum to `1.0` within a tolerance of `0.001`. This prevents subtle ranking drift caused by unconstrained weight vectors. Second, it allocates an LRU cache with a capacity of `1,000` entries. Given that the HelixTranslate production fleet currently evaluates approximately 40 active provider-model pairs, this cache comfortably retains the entire working set with a 25× headroom factor for A/B test models and seasonal provider additions. Third, the constructor stores the dependencies in the struct fields, ensuring that the engine holds no nil references after a successful return.

Code Block 1 — Scoring Engine Constructor and Weight Validation

```
type ScoringEngine struct {
    db      *sql.DB
    client  *http.Client
    weights ScoreWeights
    cache   *lru.Cache
    mu      sync.RWMutex
    logger  *log.Logger
}

func NewScoringEngine(db *sql.DB, client *http.Client, weights
        ScoreWeights, logger *log.Logger) (*ScoringEngine, error)
        {
    if err := validateWeights(weights); err != nil {
        return nil, err
    }
    cache, _ := lru.New(1000)
    return &ScoringEngine{db: db, client: client, weights:
        weights, cache: cache, logger: logger}, nil
}

func validateWeights(w ScoreWeights) error {
    sum := w.Speed + w.Cost + w.Efficiency + w.Capability +
        w.Recency
    if math.Abs(sum-1.0) > 0.001 {
        return fmt.Errorf("weights must sum to 1.0, got %.3f",
        sum)
    }
    return nil
}
```

The `sync.RWMutex` (`mu`) serialises concurrent access to the LRU cache. In practice, the cache is read-heavy — once a model's composite score is resolved, subsequent translation requests for the same language pair and provider hit the read-locked fast path. Write locks are acquired only on cache misses and during cache invalidation triggered by the `CacheTTL` expiry. The database handle (`db`) is the same pooled SQLite connection used by the benchmark store and challenge bank, so no additional connection pool tuning is required at the scoring layer.

4.2 Five-Component Score Calculation

The composite score is a weighted sum of five normalised component scores, each mapped to the range `[0, 10]`. The choice of five components reflects the operational realities of running a translation service: users demand accurate output (capability), operators must control per-token spend (cost), throughput scales with latency (speed), input-billing models reward token efficiency (efficiency), and newer models generally outperform older ones on multilingual benchmarks (recency). The mathematical formulation of each component is intentionally simple — logarithmic, linear, or ratio-based — to guarantee monotonicity, computational stability, and ease of debugging when a model's rank shifts unexpectedly.

4.2.1 Weight Comparison: HelixAgent versus HelixTranslate

Table 2 contrasts the default weight vectors shipped with the upstream LLMsVerifier project ("HelixAgent") against the values calibrated for HelixTranslate. The redistribution is the product of two months of production A/B testing across the six core language pairs supported by the translation service.

Table 2 — Weight Comparison: HelixAgent Defaults versus HelixTranslate Optimised

Component	HelixAgent Default	HelixTranslate Optimised	Rationale
Response Speed	25%	20%	Translation tolerates latency because documents are processed asynchronously; real-time chat demands faster turnaround
Cost	25%	30%	Translation workloads are token-volume heavy; a 1-cent per-million-token price delta compounds to thousands of dollars monthly at scale
Efficiency	20%	25%	Token efficiency is critical for translation because providers bill by input tokens; efficient models produce equivalent translations with fewer prompt tokens
Capability	20%	20%	Unchanged — translation quality must remain the floor constraint; no reduction in capability weight is acceptable
Recency	10%	5%	Model age matters less for translation than for general reasoning; established multilingual models retain competitiveness over longer horizons

The most significant change is the elevation of the **Efficiency** component from 20% to 25%. In HelixTranslate's workload profile, translation prompts are long (hundreds to thousands of source-language tokens), and providers charge by input token count. A model that can achieve the same BLEU-equivalent quality with 30% fewer prompt tokens — for example, by accepting a shorter system instruction or by exhibiting less instruction-following overhead — delivers a direct cost reduction proportional to that efficiency gain. The 5-percentage-point reallocation from Recency to Efficiency reflects this economic reality: a two-year-old multilingual model that produces compact translations outranks a three-month-old generalist that requires verbose prompting.

4.2.2 Response Speed Score — Logarithmic Decay

The response-speed component translates the p50 (median) end-to-end latency, measured in milliseconds from request dispatch to the first response token, into a score on `[0, 10]`. The mapping uses a negative exponential function with a characteristic time constant of `1,000` milliseconds. At `100` ms, the score is approximately `9.0`; at `1,000` ms, it falls to `3.68`; at `2,000` ms, it approaches `1.35`. This logarithmic compression ensures that ultra-low-latency providers receive diminishing score bonuses beyond the sub-200 ms threshold — a region where human perception of translation turnaround is already saturated — while high-latency providers are penalised rapidly without collapsing the score to zero.

Code Block 2 — Response Speed Score Calculator

```
func (e *ScoringEngine) CalculateResponseSpeedScore(p50LatencyMs
        float64) float64 {
    if p50LatencyMs <= 0 {
        return 10.0
    }
    score := 10.0 * math.Exp(-p50LatencyMs/1000.0) // 1s -> 3.68,
        100ms -> 9.0
    return math.Max(0, math.Min(10, score))
}
```

The guard clause `p50LatencyMs <= 0` returns the maximum score of `10.0`. This handles the edge case where a provider's benchmark data is stale or temporarily unavailable; the engine conservatively assigns the best possible speed score rather than zeroing out the component, which would unduly penalise the provider's composite ranking.

4.2.3 Cost Score — Inverse Linear

The cost component maps the normalised cost-per-million-tokens figure onto `[0, 10]` using an inverse linear relationship. At `$0` per million tokens, the score is `10.0`; at `$50` per million, it reaches `0.0`. Providers priced above `$50` per million tokens are clamped to zero, reflecting a hard policy ceiling: any model costing more than this threshold is economically infeasible for production translation at HelixTranslate's volume tier, regardless of its capability or speed.

Code Block 3 — Cost Score Calculator

```
func (e *ScoringEngine) CalculateCostScore(costPer1M float64)
        float64 {
    if costPer1M <= 0 {
        return 10.0
    }
    score := 10.0 * (1.0 - costPer1M/50.0) // $0 -> 10, $50 -> 0
    return math.Max(0, math.Min(10, score))
}
```

The cost-per-million value is computed as the weighted average of input and output token pricing, using a 3:1 input-to-output ratio that approximates HelixTranslate's actual prompt-to-completion token distribution. This ratio is parameterised in the `[verifier.pricing]` configuration stanza and may be adjusted per language pair

if empirical telemetry reveals a different distribution. The `costPer1M` parameter passed to `CalculateCostScore` is already normalised by the pricing loader, so the scoring function itself remains provider-agnostic.

4.2.4 Efficiency Score — Output-to-Input Ratio

Token efficiency is measured as the ratio of output tokens (the translated text) to input tokens (the source text plus system prompt, instructions, and any formatting overhead). For a perfectly efficient model, this ratio is `1.0` — every input token is converted to exactly one output token — yielding a score of `10.0`. Ratios below `1.0` indicate that the model "wastes" input tokens on redundant reasoning, chain-of-thought preamble, or verbose formatting, which inflates the provider's bill without improving translation quality.

Code Block 4 — Efficiency Score Calculator

```
func (e *ScoringEngine) CalculateEfficiencyScore(outputTokens,
        inputTokens int) float64 {
    if inputTokens <= 0 {
        return 5.0
    }
    ratio := float64(outputTokens) / float64(inputTokens)
    score := ratio * 10.0 // ratio 1.0 -> 10, 0.5 -> 5
    return math.Max(0, math.Min(10, score))
}
```

The guard clause for `inputTokens <= 0` returns a neutral score of `5.0`, preventing division-by-zero panics while neither rewarding nor penalising the provider. In practice, this branch is rarely hit because the benchmark pipeline always records positive token counts. The clamping ensures that pathological cases — such as a model emitting zero output tokens due to a content-policy block — do not produce negative scores.

4.2.5 Capability Score — Challenge Pass Rate

The capability component reflects the model's performance on the curated challenge bank populated in Phase 1. Each challenge in the bank has an expected answer and a grading rubric (exact match, BLEU threshold, or semantic similarity). The pass rate is the ratio of challenges passed to challenges attempted. This figure is already normalised to `[0, 1]` by the challenge runner, and the scoring function simply scales it to `[0, 10]`.

Code Block 5 — Capability Score Calculator

```
func (e *ScoringEngine) CalculateCapabilityScore(passed, total
        int) float64 {
    if total == 0 {
        return 0.0
    }
    score := float64(passed) / float64(total) * 10.0
    return math.Max(0, math.Min(10, score))
}
```

When `total == 0` — for example, when a new provider has been registered but not yet benchmarked — the function returns `0.0`. This is the only component whose default-on-absence is zero rather than neutral. The rationale is conservative: a provider with no capability data must not be promoted to production routing until at least one full challenge-bank run has completed. The zero score ensures that such a provider's composite ranking falls below the minimum threshold (`5.5` for the `"budget"` tier), keeping it out of the routing pool.

4.2.6 Recency Score — Exponential Age Decay

The recency component penalises model age using an exponential decay function with a half-life of `90` days. A model released today scores `10.0`; after 90 days, its score decays to `5.0`; after 180 days, to `2.5`; after one year, to approximately `1.25`. The half-life of 90 days was selected because it aligns with the typical cadence of major multilingual model releases (e.g., GPT-4 series, Claude family, Gemini updates). Models older than two years receive negligible recency scores but can still rank competitively if their capability, efficiency, and cost scores are sufficiently high.

Code Block 6 — Recency Score Calculator

```
func (e *ScoringEngine) CalculateRecencyScore(releaseDate
        time.Time) float64 {
    daysOld := time.Since(releaseDate).Hours() / 24
    halfLife := 90.0 // days
    score := 10.0 * math.Exp(-daysOld*math.Ln2/halfLife)
    return math.Max(0, math.Min(10, score))
}
```

The `releaseDate` is sourced from the model metadata table (`model_registry.release_date`), which is populated during Phase 2's provider registration workflow. For providers that do not publicly disclose release dates, the integration falls back to the first-seen date recorded in the verifier's own telemetry, ensuring that the recency score is always computable even when upstream metadata is incomplete.

4.3 Composite Score and Thresholds

After the five component scores have been independently calculated and clamped to `[0, 10]`, the scoring engine computes the weighted composite using the HelixTranslate-optimised weight vector. The composite score itself is also clamped to `[0, 10]`, though in practice the individual component clamping makes this a defensive measure rather than an active correction.

4.3.1 Composite Score Aggregation

The `CompositeScore` struct serves as the canonical data transfer object (DTO) for downstream consumers. It carries the model identifier, a nested `ComponentScores` struct with the five raw component values, the overall weighted score, a tier classification string, and a UTC timestamp recording when the calculation occurred.

This timestamp is critical for cache invalidation: the `ScoreAdapter` uses `CalculatedAt` to determine whether an in-memory cached score has exceeded the `CacheTTL` defined in Table 1.

The tier classification partitions models into three operational buckets:

- **Premium** (`overall >= 8.5`): High-trust models eligible for critical-translation routing (legal, medical, financial domains).
- **Standard** (`overall >= 7.0`): General-purpose models used for the majority of translation requests.
- **Budget** (`overall >= 5.5`): Acceptable models reserved for low-priority or high-volume batch jobs where cost minimisation outweighs quality maximisation.

Models scoring below `5.5` are classified as `"unranked"` and are excluded from the `ProviderSelector`'s active routing pool. They remain visible in the dashboard and API for diagnostic purposes but receive no production traffic.

Code Block 7 — Composite Score Calculation and Tier Assignment

```
type CompositeScore struct {
    ModelID      string           `json:"model_id"`
    Components   ComponentScores `json:"components"`
    Overall      float64          `json:"overall"`
    Tier         string           `json:"tier"`
    CalculatedAt time.Time        `json:"calculated_at"`
}

func (e *ScoringEngine) CalculateCompositeScore(components
        ComponentScores) (*CompositeScore, error) {
    overall := e.weights.Speed*components.Speed +
        e.weights.Cost*components.Cost +
        e.weights.Efficiency*components.Efficiency +
        e.weights.Capability*components.Capability +
        e.weights.Recency*components.Recency

    tier := "unranked"
    switch {
    case overall >= 8.5:
        tier = "premium"
    case overall >= 7.0:
        tier = "standard"
    case overall >= 5.5:
        tier = "budget"
    }

    return &CompositeScore{
        ModelID:      components.ModelID,
        Overall:      overall,
        Tier:         tier,
        Components:   components,
        CalculatedAt: time.Now().UTC(),
    }, nil
}
```

```
func IsModelQualified(score *CompositeScore, minThreshold
        float64) bool {
    return score != nil && score.Overall >= minThreshold
}
```

The `IsModelQualified` helper is invoked by the `ProviderSelector` (Chapter 5) before including a model in the ranked candidate list. It performs a nil guard — essential because `CalculateCompositeScore` may return an error if a component calculation panics — and compares the overall score against a caller-defined threshold. In HelixTranslate's default configuration, the selector uses `7.0` as the minimum threshold, meaning only `"premium"` and `"standard"` models are eligible for general routing, while `"budget"` models are reserved for explicitly marked low-priority jobs.

4.3.2 Enhanced Mode: EMA Smoothing

When the engine is configured in `"enhanced"` mode, the composite calculation in Code Block 7 is preceded by an exponential moving average (EMA) step applied to each component. The EMA formula is:

```
EMA_today = alpha * raw_score + (1 - alpha) * EMA_yesterday
```

where `alpha = 2 / (N + 1)` and `N = 7` days. This yields an `alpha` of `0.25`, meaning today's raw score contributes 25% to the smoothed value while the historical EMA contributes 75%. The EMA state is persisted in the `score_ema` table with a composite primary key of `(model_id, component_name)`, and is updated atomically within the same database transaction that records the raw score history (Section 4.5). The EMA step adds approximately 5 ms per model to the evaluation pipeline — a negligible overhead given that the entire scoring cycle completes in under 200 ms for the full provider fleet.

4.4 Score Adapter Service

The `ScoreAdapter` is the anti-corruption layer between LLMsVerifier's internal score representation and HelixTranslate's `ProviderScore` domain model. Its responsibilities are: (1) fetching the latest `CompositeScore` from the verifier's API or local cache, (2) mapping fields according to the HelixTranslate naming convention, (3) stripping verifier-internal metadata suffixes from model names, and (4) caching the adapted result to avoid redundant transformations. The adapter lives in `internal/verifier/adapter.go` and is consumed by the translation engine's `ProviderSelector` (Chapter 5) and by the management dashboard's provider status endpoint.

4.4.1 Field Mapping

Table 3 documents the complete field-level mapping from LLMsVerifier's `CompositeScore` and its nested `ComponentScores` to HelixTranslate's `ProviderScore` struct.

Table 3 — Field Mapping: LLMsVerifier to HelixTranslate

LLMsVerifier Field	HelixTranslate Field	Type	Notes
`ModelID`	`ProviderID`	string	Direct mapping; used as the routing key
`ModelName`	`ProviderName`	string	Strip `" (SC:x.x)"` suffix added by verifier for display
`OverallScore`	`OverallScore`	float64	One-to-one numeric copy
`SpeedScore`	`SpeedScore`	float64	One-to-one numeric copy
`CostScore`	`CostScore`	float64	One-to-one numeric copy
`EfficiencyScore`	`EfficiencyScore`	float64	One-to-one numeric copy
`CapabilityScore`	`CapabilityScore`	float64	One-to-one numeric copy
`RecencyScore`	`RecencyScore`	float64	One-to-one numeric copy
`LastCalculated`	`LastUpdated`	time.Time	Renamed for consistency with HelixTranslate's audit vocabulary

The mapping is almost entirely mechanical — eight of the nine fields are copied without transformation — which is by design. The scoring engine already performs the complex work of normalising, weighting, and tiering. The adapter's only semantic responsibility is the `ProviderName` sanitisation: the verifier UI appends a composite-score suffix (e.g., `" (SC:7.3)"`) to model names for dashboard readability, but this suffix must be removed before the name is exposed to end users or logged in translation audit trails.

4.4.2 Adapter Implementation

The adapter maintains its own LRU cache, separate from the scoring engine's cache, because the two caches have different keys and TTL semantics. The adapter cache is keyed by `modelID` (string) and stores `*ProviderScore` values. The `CacheTTL` from Table 1 controls how long an adapted score remains valid; when a cached entry expires, the adapter re-fetches from the verifier client rather than recomputing the score. This separation of concerns means that adapter cache misses do not necessarily trigger scoring engine recalculations — they may simply require a field-mapping pass over an already-resolved composite score.

Code Block 8 — ScoreAdapter with Caching and Field Mapping

```
type ScoreAdapter struct {
    client   *VerifierClient
    cache    *lru.Cache
    cacheTTL time.Duration
    mu       sync.RWMutex
}
```

```
func (a *ScoreAdapter) Adapt(ctx context.Context, modelID string)
        (*ProviderScore, error) {
    a.mu.RLock()
    if cached, ok := a.cache.Get(modelID); ok {
        a.mu.RUnlock()
        return cached.(*ProviderScore), nil
    }
    a.mu.RUnlock()

    score, err := a.client.GetScore(ctx, modelID)
    if err != nil {
        return nil, fmt.Errorf("fetch score for %s: %w", modelID,
        err)
    }

    adapted := &ProviderScore{
        ProviderID:      score.ModelID,
        ProviderName:    strings.TrimSuffix(score.ModelName, "
        (SC:"+fmt.Sprintf("%.1f", score.OverallScore)+")"),
        OverallScore:    score.OverallScore,
        SpeedScore:      score.Components.SpeedScore,
        CostScore:       score.Components.CostScore,
        EfficiencyScore: score.Components.EfficiencyScore,
        CapabilityScore: score.Components.CapabilityScore,
        RecencyScore:    score.Components.RecencyScore,
        LastUpdated:     score.LastCalculated,
    }

    a.mu.Lock()
    a.cache.Add(modelID, adapted)
    a.mu.Unlock()
    return adapted, nil
}
```

The `Adapt` method follows a read-through cache pattern. The read lock (`RLock`) is held only for the cache lookup, minimising contention with concurrent translation requests. If the cache misses, the lock is released before the network call to `a.client.GetScore`, preventing the adapter from holding a lock across I/O. After the client returns a `CompositeScore`, the adapter performs the field mapping, acquires a write lock, and inserts the adapted value into the cache. The cache eviction policy is LRU with a capacity of `500` entries, which is half the engine's cache capacity because the adapter typically serves a smaller subset of models — only those currently enabled for translation routing.

4.4.3 Batch Adaptation

For bulk operations — such as the dashboard's provider list endpoint or the selector's initialisation pass — the adapter exposes a `AdaptBatch` method that accepts a slice of `modelID` strings and returns a map of `modelID` to `*ProviderScore`. `AdaptBatch` uses a single read lock for the cache scan, then issues parallel client fetches for the missing entries using a worker pool sized to `min(len(missing), 10)`. This batch path reduces the aggregate latency of

adapting 40 models from approximately `40 × 15 ms = 600 ms` (sequential) to under `80 ms` (parallel with 10 workers), a critical improvement for the dashboard's cold-start render time.

4.5 Score Persistence

Longitudinal score tracking is essential for two operational functions within HelixTranslate: trend-based provider deprecation alerts and retrospective cost-quality analysis. The scoring engine records every composite score calculation to the `score_history` table, which is query-optimised for time-range scans by `model_id` and `calculated_at`. The persistence layer is intentionally simple — raw SQL `INSERT` statements executed against the shared SQLite handle — because score writes are append-only and require no complex transactional logic beyond the atomic EMA update described in Section 4.3.2.

4.5.1 Retention Policy

Score history accumulates rapidly: with 40 models, daily evaluations, and five component scores per model, the system generates approximately `73,000` rows per year. To prevent unbounded storage growth, a three-tier retention policy is applied by a background goroutine launched at engine initialisation.

Table 4 — Score History Retention Policy

Period	Retention Granularity	Action
0–90 days	All individual scores	Full granularity for operational dashboards and debugging
90–365 days	Weekly aggregates	Average composite and component scores per model per ISO week
1+ years	Monthly archives	Compressed JSON blobs moved to cold storage directory (`./data/archives/`)

The retention policy is enforced by a scheduled job that runs at 02:00 UTC daily, a time chosen to minimise interference with the 03:00 UTC benchmark cycle (Chapter 3). The job executes three SQL statements inside a single transaction:

1. **Roll-up**: For records older than 90 days but newer than 365 days, compute weekly averages grouped by `model_id` and `strftime('%Y-%W', calculated_at)`, insert the aggregates into `score_history_weekly`, and delete the corresponding raw rows from `score_history`.
2. **Archive**: For records older than 365 days, export the rows to a gzip-compressed JSON file named `score_history_YYYY-MM.json.gz` in the cold storage directory, then delete the rows from the hot table.
3. **Vacuum**: Run `VACUUM` on the SQLite database to reclaim the freed pages and reduce the on-disk footprint.

The roll-up step preserves statistical accuracy because the component scores are linear combinations, and the average of a linear combination equals the linear combination of the averages. Therefore, weekly aggregate scores can be fed back into the EMA calculation without loss of mathematical correctness.

4.5.2 History Recording

The `RecordScoreHistory` method is called synchronously at the end of every composite score calculation, ensuring that the database always contains a complete audit trail up to the most recent evaluation. The insertion includes all five component scores, the composite score, and the tier classification, enabling retrospective queries such as "show me all models that transitioned from `standard` to `budget` tier in the last 30 days" — a query that powers the provider-health alert system.

Code Block 9 — Score History Recording

```
func (e *ScoringEngine) RecordScoreHistory(score *CompositeScore)
        error {
    _, err := e.db.Exec(`
        INSERT INTO score_history
            (model_id, composite_score, component_speed,
        component_cost,
             component_efficiency, component_capability,
        component_recency, tier, calculated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`,
        score.ModelID, score.Overall,
        score.Components.Speed, score.Components.Cost,
        score.Components.Efficiency, score.Components.Capability,
        score.Components.Recency, score.Tier,
        score.CalculatedAt.UTC().Format(time.RFC3339),
    )
    if err != nil {

        e.logger.Printf("ERROR: failed to record score history for
        %s: %v", score.ModelID, err)
    }
    return err
}
```

The `calculated_at` column is stored as an RFC 3339 string to ensure consistent time-zone handling across SQLite clients. The error is both returned to the caller and logged, because a failed history write is a non-fatal event — the composite score remains valid and usable for routing — but it represents a data-loss incident that operators must be able to detect and investigate.

4.5.3 Query Interface

In addition to raw inserts, the persistence layer exposes three query functions consumed by the dashboard and alerting subsystems:

- `GetScoreHistory(modelID string, since, until time.Time) ([]ScoreHistoryRow, error)` — returns all raw or weekly-aggregated rows

for a given model in a time range, transparently routing the query to `score_history` or `score_history_weekly` based on the range endpoints.
- `GetTierTransitions(since time.Time) ([]TierTransition, error)` — returns a list of `(model_id, old_tier, new_tier, transitioned_at)` tuples for all models that changed tier classification within the specified window.
- `GetAverageScore(modelID string, window time.Duration) (ComponentScores, error)` — returns the arithmetic mean of each component score over the specified sliding window, used by the EMA pre-warming routine when a model is re-enabled after a prolonged absence.

All query functions are implemented in `internal/verifier/history_store.go` and share the same database handle as the scoring engine. They use prepared statements cached on the `sql.DB` connection pool to amortise parse-plan overhead across repeated dashboard polls.

4.6 Phase 3 Integration Checklist

Before proceeding to Phase 4 (Provider Selector Integration), the following acceptance criteria must be satisfied:

1. **Weight Validation**: `NewScoringEngine` rejects weight vectors that do not sum to `1.0 ± 0.001`.
2. **Component Monotonicity**: Each of the five component calculators is verified to produce monotonically non-increasing scores as the input metric degrades (higher latency, higher cost, lower efficiency, lower pass rate, older release date).
3. **Tier Assignment**: A model with a synthetic perfect-component profile (`Speed=10, Cost=10, Efficiency=10, Capability=10, Recency=10`) receives `"premium"` tier; a model with all zeros receives `"unranked"`.
4. **Adapter Round-Trip**: The `ScoreAdapter` produces a `ProviderScore` whose fields match the original `CompositeScore` after accounting for the name-suffix stripping.
5. **History Persistence**: After 30 days of continuous operation, the `score_history` table contains exactly one row per model per day, and no score data older than the retention window remains in the hot table.
6. **Cache Correctness**: The combined hit rate of the engine cache and adapter cache exceeds 85% under a simulated production load of 1,000 translation requests per minute across 40 models.

Completion of this checklist confirms that the scoring pipeline is ready to feed ranked provider data into the selector and scheduler layers documented in the following chapters.